XGBoost

2026-04-17

Introduction

XGBoost (eXtreme Gradient Boosting), introduced by Chen and Guestrin in 2016, extends classical gradient boosting with L1 and L2 regularisation of leaf weights, a second-order Taylor expansion of the loss, sparsity-aware split finding, and histogram-based methods that scale to billions of rows. It has dominated competitive machine-learning benchmarks on structured tabular data for nearly a decade and remains the strongest single off-the-shelf model for most regression and classification problems on tables.

Prerequisites

A working understanding of gradient boosting, decision trees, regularisation (L1/L2 penalties), and cross-validation for hyperparameter tuning.

Theory

The XGBoost objective is

$$\mathcal{L} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2,$$

where L is the per-observation loss (squared error, log-loss, etc.), f_k is the k-th regression tree, T is the number of leaves, w are the leaf weights, γ penalises tree complexity, and λ regularises leaf weights. A second-order Taylor expansion of L yields analytic optimal leaf weights and a tractable split score.

Key hyperparameters: `eta` (learning rate), `max_depth`, `min_child_weight`, `lambda` and `alpha` (L2 and L1 leaf-weight regularisation), `gamma` (minimum loss reduction for a split), `subsample` (row sampling), `colsample_bytree` (column sampling at tree level).

Assumptions

Dataset is tabular; labels suit the chosen objective (binary, multiclass, count, regression); sufficient sample size to support tree-based interactions and depth.

R Implementation

```
library(xgboost)

set.seed(2026)
n <- 600
X <- matrix(rnorm(n * 5), n, 5)
y <- as.numeric(plogis(X[, 1] + 0.5 * X[, 2]^2) > runif(n))

idx <- sample(n, n * 0.8)
d_tr <- xgb.DMatrix(X[idx, ], label = y[idx])
d_te <- xgb.DMatrix(X[-idx, ], label = y[-idx])
```

```r
params <- list(objective = "binary:logistic", eta = 0.05,
               max_depth = 4, subsample = 0.8,
               colsample_bytree = 0.8,
               eval_metric = "logloss")

bst <- xgb.train(params = params, data = d_tr,
                 nrounds = 500,
                 watchlist = list(val = d_te),
                 early_stopping_rounds = 20,
                 verbose = 0)

cat("Best iter:", bst$best_iteration,
    "  best logloss:", round(bst$best_score, 4), "\n")

xgb.importance(model = bst)[1:5, ]
```

Output & Results

`xgb.train()` returns a fitted `xgb.Booster` with the early-stopping iteration count, best validation metric, and tree structure. `xgb.importance()` ranks features by gain (the average loss reduction attributed to splits on that feature), cover, or frequency.

Interpretation

A reporting sentence: "XGBoost with `eta = 0.05`, `max_depth = 4`, row and column subsampling at 0.8, converged after 217 boosting rounds with early stopping at 20-round patience; validation log-loss 0.38, ROC AUC 0.92. Feature-gain importance identified x_1 and x_2 as the strongest contributors, consistent with the simulated data-generating process." Always state the hyperparameters, early-stopping settings, and validation metrics.

Practical Tips

- Always use early stopping (`early_stopping_rounds`) to avoid over-training; without it, XGBoost trains until `nrounds` regardless of whether the validation loss has plateaued.
- Choose `objective` to match the target: `"reg:squarederror"` for regression, `"binary:logistic"` for binary classification, `"multi:softprob"` for multiclass, `"count:poisson"` for counts, `"survival:cox"` for time-to-event.
- Tune with Bayesian optimisation (`ParBayesianOptimization`, `tune::tune_bayes()`) rather than grid search; XGBoost has many correlated knobs and grid search wastes evaluations.
- `tree_method = "hist"` accelerates training 5–10× on large data; `"gpu_hist"` adds another order of magnitude on GPU.
- Monotone constraints (`monotone_constraints`) enforce domain-knowledge directions for predictor-outcome relationships; useful in regulated industries (credit risk, healthcare).
- For interpretation beyond feature importance, use SHAP values (`xgboost::predict(..., predcontrib = TRUE)` or the `shapviz` package); SHAP gives per-prediction explanations.

R Packages Used

`xgboost` for the canonical R interface; `parsnip::boost_tree()` for the tidymodels wrapper; `lightgbm` and `catboost` as competitive alternatives; `shapviz` and `iml` for SHAP-based interpretation; `ParBayesianOptimization` for efficient hyperparameter tuning.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis